Advanced Visual Programming

Lab

Assignment No 02

Submitted To:

Sir Uzair

Submitted By:

Momina Waqar

232201039

BSCS 6 A

BS in Computer Science,

K.I.C.S.I.T , Kahuta

30-March-2026

March 30, 2026

Contents

1 Introduction

1.1 Problem Statement

In this project, the objective is to create a mini AI-based application that predicts the sentiment of user input text as Positive, Negative, or Neutral using .NET and ML.NET.

1.2 Objective

The application should:

- Accept user text input.
- Predict sentiment using a trained ML model.
- Display the sentiment clearly to the user.

2 System Design

2.1 Architecture Diagram

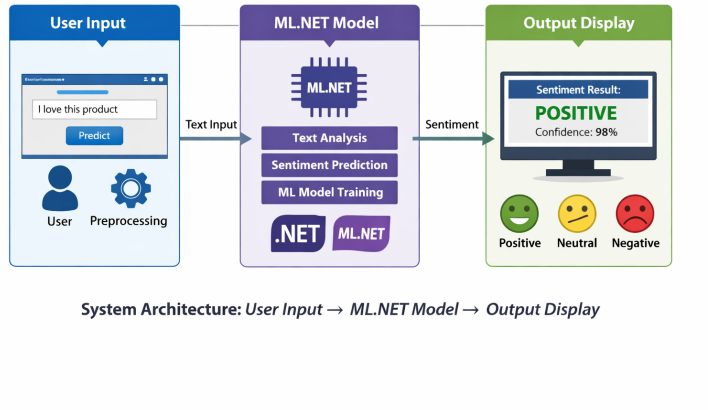


Figure 1: System Architecture: User Input → ML.NET Model → Output Display

2.2 Workflow

1. User enters text in the TextBox.

2. Clicks the Predict button.
3. Text is processed and analyzed.
4. Keyword matching is performed.
5. ML model predicts sentiment if required.
6. Result is displayed to the user.

3 Tools & Technologies

- **.NET Framework:** 4.8
- **Machine Learning Library: :contentReference[oaicite:0]index=0**
- **Algorithm:** FastTree Binary Classification
- **IDE:** Visual Studio 2022

4 System Implementation

4.1 Data Generation

- 2500 positive and 2500 negative sentences are generated.
- Templates are used such as:
  - "I really {word} this"
  - "This is very {word}"
- Each sentence is labeled:
  - True → Positive
  - False → Negative

4.2 Model Training

- Text is converted into features using FeaturizeText.
- FastTree classifier is used.
- Model is trained using generated dataset.

```
private void TrainModel()
{
    mlContext = new MLContext();

    var trainData = mlContext.Data.LoadFromEnumerable(GetTrainingData());

    var pipeline = mlContext.Transforms.Text.FeaturizeText(
                        outputColumnName: "Features",
                        inputColumnName: nameof(SentimentData.Text))
                    .Append(mlContext.BinaryClassification.Trainers.FastTree(
                        labelColumnName: nameof(SentimentData.Label),
                        featureColumnName: "Features"));

    var model = pipeline.Fit(trainData);

    engine = mlContext.Model.CreatePredictionEngine<SentimentData, SentimentPrediction>(model);
}
```

5 Code Snippets

5.1 Training Model

5.2 Prediction Logic

1. Keyword Matching:
    - Input is split into words.
    - Words are matched with positive and negative word lists.
    - If positive ¿ negative → Positive
    - If negative ¿ positive → Negative
2. Neutral Detection:
    - If no keywords found → Neutral
3. ML Model:
    - Used when keywords are unclear or mixed.
    - Provides probability-based prediction.

```
private void btnPredict_Click(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(textBox1.Text))
    {
        MessageBox.Show("Please enter text!");
        return;
    }

    string inputText = textBox1.Text.Trim().ToLower();

    // WORD SPLIT
    var words = inputText.Split(' ');

    int positiveCount = words.Count(w => positiveWords.Contains(w));
    int negativeCount = words.Count(w => negativeWords.Contains(w));
```

```
if (positiveCount == 0 && negativeCount == 0)
{
    lblMessage.Text = "Neutral 😐";
    return;
}

// Strong keyword decision
if (positiveCount > negativeCount)
{
    lblMessage.Text = "Positive 😊";
    return;
}
else if (negativeCount > positiveCount)
{
    lblMessage.Text = "Negative 😞";
    return;
}


    var input = new SentimentData { Text = inputText };
    var result = engine.Predict(input);

    lblMessage.Text = result.Prediction
        ? $"Positive 😊 ({result.Probability:P2})"
        : $"Negative 😞 ({result.Probability:P2})";
}
```

6 Application Preview

6.1 UI

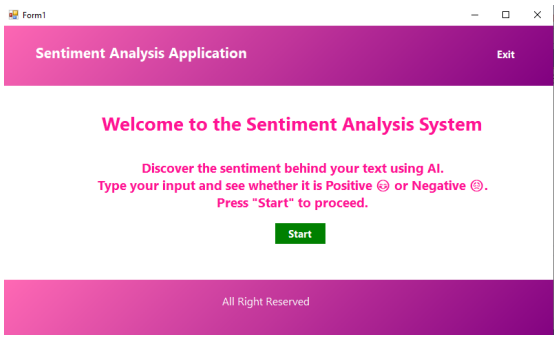
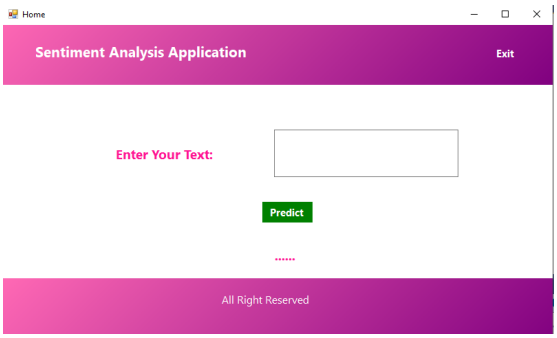


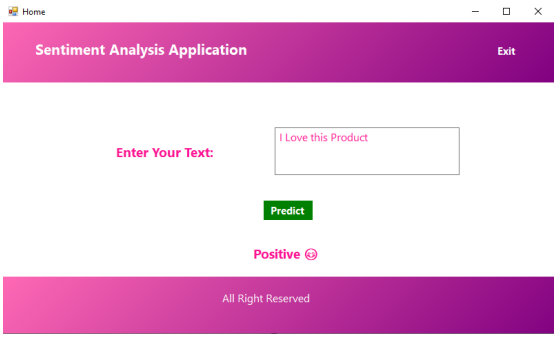
Figure 2: Welcome Screen UI



Home UI

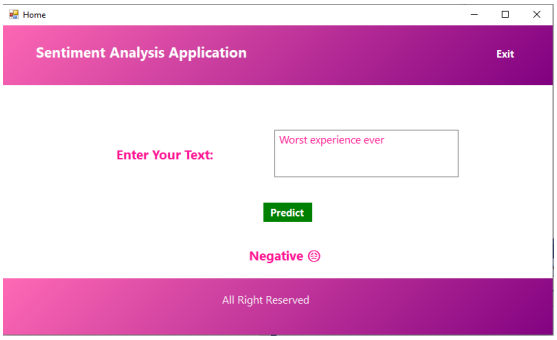
6.2 Outputs

Positive



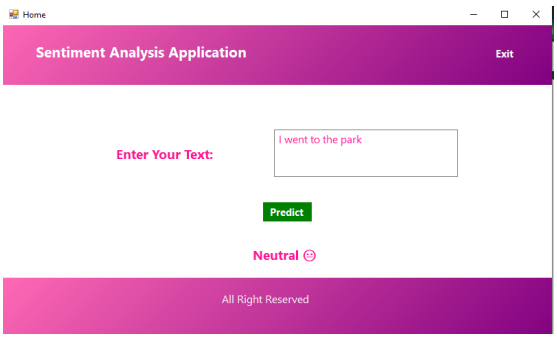
Positive Output Preview

Negative



Negative Output Preview

Neutral



Neutral Output Preview

6.3 Working of the Application

This application is a hybrid AI-based sentiment analysis system developed using **ML.NET**. The working of the system is divided into two main parts: keyword-based analysis and machine learning prediction.

When the application starts, the machine learning model is trained using a dataset of generated sentences labeled as positive and negative. The model uses the FastTree algorithm to learn patterns from the data.

When a user enters text in the input box and clicks the Predict button, the application first processes the text by converting it into lowercase and splitting it into individual words. It then compares these words with predefined lists of positive and negative keywords.

If the number of positive words is greater than negative words, the sentiment is classified as Positive. If negative words are greater, the sentiment is classified as Negative. If no known keywords are found, the result is Neutral.

In cases where the sentiment is unclear or both positive and negative keywords are present, the trained ML model is used to make the final prediction. The model also provides a probability value indicating the confidence of the prediction.

The result is then displayed on the user interface as Positive, Negative, or Neutral.

7 Testing

Input	Predicted Output
I love this product	Positive
This is terrible	Negative
I went to the park	Neutral
Absolutely fantastic app	Positive
Worst experience ever	Negative

Table 1: Sample Inputs and Outputs

8 Results & Discussion

- The system successfully predicts sentiment.
- Keyword-based logic is fast and accurate.
- ML model handles complex cases.
- Limitation: Dataset is synthetic.

9 Docker Implementation

- The application is containerized using Docker.
- Docker ensures consistent environment across systems.
- The Dockerfile builds and runs the .NET application.

```
FROM mcr.microsoft.com/dotnet/sdk:7.0

# Set working directory inside container
WORKDIR /app

# Copy all project files into container
COPY . .

# Build the project inside container
RUN dotnet build

# Command to run the application
CMD ["dotnet", "run"]
```

10 Future Improvements

- Use real-world datasets.
- Improve model accuracy.

- Add advanced NLP preprocessing.
- Enhance UI with visualization.

11 Conclusion

This project demonstrates an AI-based sentiment analysis system using ML.NET. It combines rule-based logic with machine learning, making it a hybrid intelligent system capable of predicting sentiment effectively.

THE END